TECHNISCHE UNIVERSITÄT DRESDEN

ComNets
Deutsche Telekom Chair of Communication Networks

Faculty of Electronic and Information Institute of Communication Engineering
Chair of Communication Networks

Master Thesis

Randomized Identification Codes with a Focus on K-Identification

Chongyang Zheng

Born on: 10th October 1997 in Xi'an
Course: Nanoelectronic Systems
Matriculation number: 5198083
Matriculation year: 2023

12th January 2026

Supervisors
Juan A. Cabrera
Caspar von Lengerke

Supervising professor
Prof. Dr. h.c. Fitzek

TECHNISCHE UNIVERSITÄT DRESDEN

Fakultät Elektrotechnik und Informationstechnik
Institut für Nachrichtentechnik
Deutsche Telekom Professur Kommunikationsnetze
Prof. Dr.-Ing. Dr. h. c. Frank Fitzek

Task for Master Thesis of

Mr. Chongyang Zheng Academic year: ET/ , matriculation number:

Topic: Randomized Identification Codes with a Focus on K-Identification

Task description:

Identification via channels (ID) asks receivers to decide whether a specific message was sent, rather than to reconstruct it. Randomized ID codes achieve doubly-exponential message sets in blocklength and enable vanishing Type-I (false-accept) and Type-II (miss) error probabilities at positive ID rate. K-Identification (K-ID) generalizes this: the receiver asks whether the transmitted message belongs to a set of K candidates. K-ID is relevant for goal/semantics-driven communication (e.g., digital twins), where the action depends only on whether the current state falls into a small set of actionable hypotheses, not on exact message recovery. This thesis will study randomized ID codes with emphasis on K-ID, and extend the ideas and evaluation from “Stopping the Data Flood: Post-Shannon traffic reduction in digital-twins applications” to the K-ID setting. The work should balance theory, algorithms, and software-driven experiments using our provided library.

The student is expected to:

Survey the state of the art:

- Core ID concepts: randomized encoders/acceptance regions, ID capacity, error exponents, and operational interpretations; contrast with Shannon transmission.
- Define K-ID precisely (membership in a K-set) and position it vs. related notions (multiple-object ID, generalized ID; deterministic vs randomized variants.

Formulate the problem & metrics for K-ID:

- Formal model (channel class, randomization/common randomness budget, decoder tests).
- Report both Type-I/Type-II errors and computational metrics (encode/decode time, seed length, memory), plus finite-blocklength behavior.

Replicate & extend the digital-twins baseline to K-ID:

- Reproduce key experiments/results from the “Stopping the Data Flood” study (baseline ID) with our software stack.
- Design K-ID experiments that reflect digital-twin quantifying traffic reduction vs. reliability/latency.

Use our software library to build a modular K-ID test bench:

- Implement randomized ID/K-ID encoders and decoders (acceptance region tests) as pluggable components.

Design and run a simulation campaign:

- Measure KPIs such as: Type-I/II error curves, empirical exponents, queries per symbol, and compute/memory usage. Quantify traffic reduction vs. reliability/latency

Deliver a comparative assessment with clear takeaways:

- When do systems benefit from K-ID?
- Tradeoffs among randomization, complexity, and reliability.

Supervisors: Juan A. Cabrera, Caspar von Lengerke
Started at: 01.09.2025
To be submitted by: 11.02.2026

Prof. Dr.-Ing. Dr. h.c. Fitzek
Responsible Tutor

TECHNISCHE UNIVERSITÄT DRESDEN

ComNets
Deutsche Telekom Chair of Communication Networks

Faculty of Electronic and Information Institute of Communication Engineering

Chair of Communication Networks

Statement of authorship

I hereby certify that I have authored this document entitled *Randomized Identification Codes with a Focus on K-Identification* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. During the preparation of this document I was only supported by the following persons:

Juan A. Cabrera
Caspar von Lengerke

Additional persons were not involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, 12th January 2026

Chongyang Zheng

TECHNISCHE UNIVERSITÄT DRESDEN

ComNets
Deutsche Telekom Chair of Communication Networks

Faculty of Electronic and Information Institute of Communication Engineering

Chair of Communication Networks

Abstract

Identification via channels (ID) asks a receiver to decide whether a specific hypothesis about the transmitted message is true, rather than reconstructing the message itself. This document provides precise definitions, key capacity results, error metrics, and an experimental plan tailored to K-Identification (K-ID), where the receiver asks whether the message belongs to a set of K candidates. It contrasts randomized vs. deterministic (DI) variants, formulates problem and metrics, and outlines a modular test bench and simulation campaign aligned with the provided software stack.

Contents

List of Figures

List of Tables

Acronyms

CPS	Cyber-Physical Systems	1
DT	Digital Twin	1
ID	Identification	2
K-ID	K-Identification	1, 2
URLLC	Ultra-Reliable Low-Latency Communications	1

1 Introduction

Digital Twins (DTs) have become a standard component in modern industrial automation for connecting physical assets with their virtual counterparts. By maintaining accurate virtual copies of systems like industrial robots, Digital Twin (DT)s enable real-time monitoring, predictive maintenance, and remote control. In time-critical Cyber-Physical Systems (CPS) applications, the utility of a DT is strictly defined by the synchronization accuracy between the physical robot and the digital model [3]. Consequently, ensuring state consistency typically demands Ultra-Reliable Low-Latency Communications (URLLC) capabilities to support high-frequency updates.

However, meeting these requirements via standard periodic transmission is resource-intensive. Since the DT often possesses a precise estimate of the robot's state, continuously transmitting raw telemetry data creates significant redundancy. This inefficiency consumes scarce bandwidth without providing new information gain.

To mitigate this redundancy, recent research proposes *identification-based (ID)* synchronization [1]. Unlike classical Shannon-type communication, which aims to reconstruct the full message, ID schemes focus on a verification task: confirming whether the remote state matches a local hypothesis. Building on this concept, prior works [2] demonstrate that transmitting a compressed "verifier" (e.g., a short hash or randomized code) instead of the full state significantly reduces communication traffic. This approach effectively supports goal-oriented communication in low-latency scenarios.

However, implementing ID synchronization in real-time control loops reveals two critical engineering trade-offs. First, regarding latency, the computational overhead of encoding on edge devices may exceed the time saved by reducing transmission traffic. This shift from a communication-bound to a computation-bound bottleneck necessitates a rigorous end-to-end feasibility analysis. Second, reliability becomes a primary concern when moving from exact to semantic verification. In practical control, enforcing bitwise synchronization (standard ID) is often overly conservative, triggering unnecessary updates for benign deviations such as sensor noise. To address this, we extend identification to semantic "safe ranges" (**K-ID**), where the system verifies set membership rather than a unique value. This relaxation, however, introduces collision risks, specifically Type II (miss) errors, where dangerous desynchronizations are incorrectly classified as safe. Therefore, quantifying the relationship between the semantic acceptance range and safety risks is essential.

To address these limitations, this thesis presents a complementary study to existing ID frameworks, extending the analysis from pure traffic optimization to an **integrated view** of latency and reliability in K-Identification (K-ID) systems. The main contributions are:

First, we develop an end-to-end latency model combining computational and transmission delays. We compare cryptographic (SHA-256) and algebraic (RS-ID) encoding schemes to determine the quantitative thresholds (break-even points) where ID yields a net latency benefit as a function of bandwidth and verifier length.

Second, we formalize the K-ID problem for robotic monitoring, distinguishing *safe* from *unsafe* desynchronization via semantic safety sets. We derive quantitative bounds on miss error probabilities induced by K-ID mechanisms.

Third, we conduct a **systematic evaluation** using a reproducible software stack. Through Monte Carlo experiments on Gaussian robot trajectories, we expose a non-monotonic dependence between the semantic acceptance range (Set B) and error rates. These results provide design guidelines for balancing semantic tolerance with safety detection in edge-based DT networks.

Relation to prior work. While Identification (ID) rests on a solid information-theoretic foundation, engineering studies have primarily focused on traffic suppression. We extend this line of work by **treating latency as a primary metric** alongside traffic and by generalizing from single identification to K-ID, which aligns better with real-world DT workflows. Our results aim to inform design choices regarding *when to use ID/K-ID and with which verifier lengths* under realistic bandwidth conditions.

Organization. Chapter 2 introduces the background and related work; Chapter 3 details the system model and measurement protocol; Chapter 4 describes the software stack and reproducibility; Chapter 5 presents the experimental results, including latency–traffic curves parameterized by bandwidth and load; Chapter 6 concludes and outlines future engineering directions for K-ID.

2 Background and Related Work

This chapter establishes the theoretical foundations and reviews the state-of-the-art literature relevant to this thesis. We first outline the operational constraints of Digital Twins in Cyber-Physical Systems using recent studies on remote robot control [3]. Subsequently, we delve into the information-theoretic principles of Identification (ID) [**ahlswede1989**] and its application in Post-Shannon communication [1]. Finally, we discuss practical encoding schemes—specifically algebraic and cryptographic approaches—and the extension to K-Identification (K-ID) for semantic event recognition [2], identifying the specific latency and reliability gaps this thesis aims to bridge.

2.1 Digital Twin Synchronization in CPS

Digital Twins (DTs) serve as high-fidelity virtual replicas of physical assets, essential for applications requiring real-time interaction over long distances. As demonstrated by Tsokalo et al. [3], in scenarios such as remote robot control with human-in-the-loop, the DT acts as a predictor to mask network delays. However, the effectiveness of this setup is strictly bounded by the synchronization latency between the physical robot and the DT. High latency or jitter can lead to inconsistencies between the physical and virtual states, potentially causing instability in the control loop or safety violations [3].

Standard synchronization protocols typically employ periodic transmission of raw telemetry data. While robust, this approach ignores the semantic content of the data, leading to the "data flood" phenomenon described in [1], where redundant updates consume scarce wireless bandwidth without adding information value.

2.2 Identification-based (ID) Synchronization

To mitigate bandwidth inefficiencies, recent research has pivoted from classical data transmission to *Identification-based* synchronization.

2.2.1 Application in Digital Twins

von Lengerke et al. [1] proposed a "Post-Shannon" traffic reduction scheme for DT applications. Their work demonstrates that by transmitting compressed, randomized "verifiers" (hash-based signatures) instead of raw data, the communication traffic can be drastically reduced while

maintaining synchronization accuracy. This approach exploits the fact that the DT often holds a correct local prediction; thus, the communication task simplifies to a hypothesis test (Match/No-Match) rather than full data reconstruction [1].

2.3 Theoretical Foundations of Identification

The theoretical basis for this paradigm lies in the seminal work of Ahlswede and Dueck [**ahlswede1989**] on "Identification via Channels." This section details why ID fundamentally differs from transmission.

2.3.1 The Double Exponential Capacity Phenomenon

In Shannon's classical transmission theory, the goal is to distinctively reconstruct a message m. This requires mapping each message to a **disjoint** decoding set in the output space. Consequently, the number of messages M is bounded by the volume of the space, scaling exponentially with blocklength n (i.e., $M \sim 2^{nR}$) [**ahlswede1989**].

In contrast, the identification problem asks: "Is the current state equal to i?". Ahlswede and Dueck proved that if we relax the disjointness constraint and allow decoding sets to overlap (provided their intersection is statistically small), the capacity changes fundamentally [**ahlswede1989**].

Intuitive Explanation: The difference can be understood through set theory. In transmission, we map messages to elements (points) in the space. In identification, we map messages to subsets (shapes) of the space. Since the number of subsets (the power set) is 2^{elements}, the number of identifiable messages N scales doubly exponentially:

$$N \sim 2^{2^{nC}} \tag{2.1}$$

This implies that for verifying synchronization states, an extremely short verifier length n suffices to distinguish a vast, practically infinite number of potential robot states.

2.3.2 Why Randomization is Mandatory

A critical caveat in ID theory is that deterministic coding cannot achieve this double exponential growth. For any deterministic mapping of N messages into overlapping sets, there inevitably exists a "worst-case pair" of messages whose decoding sets overlap significantly, leading to a collision probability $P_e \approx 1$ [**ahlswede1989**].

To mitigate this, **Randomization** (or Common Randomness) is mandatory. In a randomized scheme, the mapping depends on a stochastic variable r (seed/salt). This "smears" the collision probability across the entire message space, ensuring that for *any* pair of distinct states, the probability of collision is uniformly bounded by a small ϵ.

2.4 Semantic Event Recognition: K-Identification (K-ID)

While standard ID focuses on distinguishing a unique state, practical IoT and control applications often exhibit "event recognition behavior," where the goal is to identify whether the system state belongs to a specific target set. This generalization is known as **K-Identification (K-ID)**.

2.4.1 The Concept of Set Identification

Salariseddigh et al. [2] recently formalized Deterministic K-Identification (DKI). They highlight that in many smart systems, the established Shannon capacity is not the central metric; instead, the system aims to identify a goal message set of size K [2]. Their results on Binary Symmetric Channels indicate that the identifiable message sets can grow exponentially with codeword length.

2.4.2 K-ID in Robotic Safety Monitoring

Adapting this concept to robotic control, this thesis defines K-ID as the process of verifying whether the robot's state lies within a semantic "safe range" (denoted as Set $\mathcal{B}$). Unlike the deterministic models in [2], we apply K-ID in a randomized encoding framework to handle continuous Gaussian state distributions.

2.5 Practical Encoding Schemes

Translating theoretical randomized ID codes into practical edge-computing algorithms requires specific construction methods. We analyze two primary schemes used in our latency modeling.

2.5.1 Algebraic Encoding: Reed-Solomon Identification (RS-ID)

RS-ID constructs the verifier using polynomial evaluation over a finite field $\mathbb{F}_q$. Let the robot's state S be represented as a vector of coefficients $S = [s_0, \ldots, s_k]$. We define a polynomial $P_S(x) = \sum s_j x^j$. The verifier v is the evaluation of this polynomial at a random point r (the common randomness):

$$v = P_S(r) = \sum_{j=0}^{k} s_j r^j \pmod{q} \tag{2.2}$$

Property: Based on the fundamental theorem of algebra, two distinct polynomials of degree k intersect at most at k points. Thus, the collision probability is strictly bounded by k/q. While theoretically secure, this operation incurs high computational latency ($O(k \log k)$ or $O(k^2)$).

2.5.2 Cryptographic Hashing: Truncated SHA-256

For engineering efficiency, we utilize cryptographic hash functions. The verifier is constructed by concatenating the state S with a time-variant salt r, hashing, and truncating to n_{ver} bits:

$$v = \text{Truncate}_{n_{\text{ver}}}(\text{SHA-256}(S \mid\mid r)) \tag{2.3}$$

Property: The salt r provides the necessary randomization. Assuming the hash behaves as a random oracle, the collision probability approximates $1/2^{n_{\text{ver}}}$. This method benefits from hardware acceleration on modern CPUs but lacks the strict algebraic proofs of RS-ID.

2.6 Summary and Research Gaps

Despite the progress in traffic reduction [1] and K-ID theory [2], two critical engineering gaps remain:

1. **Latency Trade-offs:** Previous works like [1] focus primarily on traffic reduction. However, the *computational latency* introduced by encoding algorithms (Section 2.5) has not been modeled end-to-end. It remains unclear under what bandwidth conditions the computational cost of RS-ID or SHA-256 outweighs the transmission gain.
2. **Reliability of Randomized K-ID:** While [2] provides bounds for deterministic K-ID, the safety risks (Type II/Miss errors) of using *randomized* K-ID for continuous robotic states with varying safety range sizes (Set $\mathcal{B}$) lack empirical characterization.

This thesis aims to address these specific limitations.

3 Design Methodology

3.1 Overview

This chapter gives the modeling and experimental methodology used to evaluate identification (ID) and its extension to K-Identification (K-ID). It defines the latency and traffic models, the measurement protocols used to obtain per-operation compute times, and the Monte-Carlo procedures used to estimate rare false-accept/miss probabilities. The goal is to state assumptions and reproducible procedures so that results in Chapter 5 can be reproduced from the provided scripts and data.

3.2 Notation and Goals

We study systems that implement a short verifier (tag) check and an optional bulk transfer upon mismatch. The central quantities are:

- L_{ver}: verifier/tag length (bits);
- L_{data}: payload length (bits), equivalently n_{data} in bytes where noted;
- p_{desync}: probability that the receiver's stored state is desynchronized and hence bulk transfer is required;
- p_{miss}: per-object false-accept probability of the verifier (i.e., when a tag incorrectly accepts a different object); we will sometimes denote the empirically measured value as $\hat{p}_{\text{miss}}$ or $p_{\text{miss,emp}}$;
- T_{encode} and T_{verify}: wall-clock compute times for one encode (sender) and one verify (receiver) operation, measured on the target platform.

The engineering objective is to quantify expected latency and traffic per object/query as functions of verifier length, number of verifications in a batch (K), and network bandwidth B.

3.3 Encoding Process

This section details the mathematical steps of the encoder families used in our evaluation and links them to the concrete implementations in the repository under `src/idsys/core/idsystems.py`.

3.3.1 SHA256ID

Given a message $M = (m_0, m_1, \ldots, m_{n-1})$ of bytes, the SHA256ID encoder computes

$$H = \mathrm{SHA256}(M) \in \{0,1\}^{256},$$

and derives a tag by truncation to e bits corresponding to the configured Galois-field exponent e:

$$\mathrm{tag} = H[0:e] \in \{0,1\}^e \equiv \mathbb{F}_{2^e} \text{ symbol},$$

where $H[0:e]$ denotes the first e bits of the digest. In our code, this is implemented by `SHA256IDEncoder.encode` which calls `idcodes.sha256id(message, gf_exp)` and returns a single integer tag representing the e-bit truncation. Verification recomputes the same mapping and compares equality (`SHA256IDVerifier.verify`).

3.3.2 RSID

Let e be the Galois-field exponent and $\mathbb{F}_{2^e}$ the finite field constructed by `idcodes.generate_gf_outer(e)`. We interpret the input bytes as field symbols $M = (x_0, x_1, \ldots, x_{n-1}) \in \mathbb{F}_{2^e}^n$.

RSID computes a Reed–Solomon codeword

$$\mathbf{C} = (c_0, \ldots, c_{n-1}) \in \mathbb{F}_{2^e}^n, \quad c_i = P(\alpha_i),$$

by evaluating a polynomial $P(X)$ of degree $< k$ (message polynomial) at evaluation points $(\alpha_0, \ldots, \alpha_{n-1})$ in $\mathbb{F}_{2^e}$. The RSID *tag* is then the symbol at a configured position $t \in \{0, \ldots, n-1\}$:

$$\mathrm{tag}(M; t, e) = c_t = P(\alpha_t) \in \mathbb{F}_{2^e}.$$

In practice, we use a compact implementation via the `idcodes` library: `RSIDEncoder.encode` iterates the configured positions `tag_pos` and calls `idcodes.rsid(message, tag_pos, gf_exp)` to return one integer per position (each representing an e-bit field symbol). The verifier recomputes these symbols and checks equality across positions (`RSIDVerifier.verify`).

Remarks:

- The field initialization is performed by `get_idcodes_instance` and `generate_gf_outer(e)` for $e \leq 16$ as seen in `RSIDEncoder`.
- For multi-tag configurations, RSID returns a list of tags $(c_{t_1}, \ldots, c_{t_r})$; verification requires all positions to match.

3.3.3 RS2ID

RS2ID is a concatenated Reed–Solomon construction that maps a byte message M to a single tag by composing an inner RS mapping with an outer RS mapping. Let e_0 denote the base Galois-field exponent. The implementation uses an effective field size $2e_0$ for the concatenated symbol, i.e., arithmetic in $\mathbb{F}_{2^{2e_0}}$ for the outer layer, while the inner layer uses $\mathbb{F}_{2^{e_0}}$ symbols.

Conceptually, the encoder forms an inner RS stream from M and packs inner symbols into an outer message polynomial $P_{\text{out}}(X)$ over $\mathbb{F}_{2^{2e_0}}$. The RS2ID tag is the outer evaluation at a configured position t_{out} with a configured inner position t_{in} governing which inner evaluation participates in the packing:

$$\text{tag}(M; t_{\text{out}}, t_{\text{in}}, e_0) \;=\; P_{\text{out}}(\alpha_{t_{\text{out}}}^{\text{out}}) \in \mathbb{F}_{2^{2e_0}}.$$

In code, `RS2IDEncoder.encode` returns a single integer tag by calling

$$\textit{extttidcodes.rs2id}(\,\textit{message}, \textit{tag\_pos}, \textit{tag\_pos\_in}, \textit{gf\_exp}\,).$$

Here `gf_exp` is set to $2e_0$ internally. Field initialization follows the backend's requirements: for small exponents, both outer and inner fields are generated via `generate_gf_outer(.)` and `generate_gf_inner(.)`. Verification recomputes the tag with the same parameters and compares equality (`RS2IDVerifier.verify`).

3.4 Latency and Traffic Model

We adopt a simple, separable latency model that decomposes total expected latency into compute and transfer components.[1] For a single verifier check (no batching), the expected latency when querying an object is

$$L_{\text{total}} = T_{\text{comp}} + T_{\text{net}},$$

where the compute term (with the simplifying assumption $T_{\text{verify}} \approx 0$) is

$$T_{\text{comp}} = T_{\text{encode}},$$

and the network term accounts for the short tag transfer and the conditional bulk transfer upon mismatch. Under a perfect (no-loss) channel and assuming negligible tag size relative to L_{data}, we approximate the network contribution as

$$T_{\text{net}} = \frac{n_{\text{ver}}}{B} + p_{\text{desync}}(1 - p_{\text{miss}})\frac{n_{\text{data}}}{B},$$

where n_{ver} is the verifier size in bytes and B is bandwidth in bytes/second. The first term models the (small) verifier transfer; the second term is the expected conditional bulk transfer cost (only when a true desync occurs and the verifier does not falsely accept). This formula matches the user formula used in our experiments and is the basis for the latency plots in Chapter 5.

[1] In our target scenario, the verify step is constant-time and negligible relative to encode and network transfer; unless noted otherwise we set $T_{\text{verify}} \approx 0$ so the compute term reduces to T_{encode}. For completeness, Section 3.4.1 retains the general form with T_{verify}.

Traffic model. Following prior "stopping the dataflow" traffic analysis, the expected per-message traffic (in bytes) can be written as

$$\text{traffic} = n_{\text{ver}} + p_{\text{desync}}(1 - p_{\text{miss}})\, n_{\text{data}} + (1 - p_{\text{desync}})\, p_{\text{FA}}\, n_{\text{data}}, \tag{3.1}$$

where p_{FA} denotes the false-accept probability while remaining synchronized. Under the perfect-channel assumption used in our experiments ($p_{\text{FA}} = 0$), this simplifies to

$$\text{traffic} = n_{\text{ver}} + p_{\text{desync}}(1 - p_{\text{miss}})\, n_{\text{data}}. \tag{3.2}$$

These traffic expressions directly induce the network term via division by throughput B (i.e., traffic$/B$), and are consistent with the latency formulation above.

On the verification cost T_{verify}. In our target setting, the verify step is a constant-time check of a small tag (e.g., testing a hash key's presence in a dictionary). Its cost is negligible compared to encode and network transfer. Hence for single-object ID we approximate $T_{\text{verify}} \approx 0$ without affecting conclusions. For K-ID, batch verify cost can be written $T^{(K)}_{\text{verify}} = \alpha + \beta K$, but each candidate's check remains constant-time and (α, β) are small in our implementation, making $T^{(K)}_{\text{verify}}$ negligible relative to T_{encode} and the network term. We use this approximation in the results chapter and support it with measurements.

For K-ID (an OR over a set of size K), the OR-aggregation changes the effective false-accept rate. If per-object false-accept probability is p and errors are approximately independent across objects, the false-accept probability for the K-set is

$$\lambda_2^{(K)} = 1 - (1 - p)^K \approx 1 - e^{-pK}.$$

Hence, in a K-verification batch, only when the batch fails to find the transmitted object is the costly bulk transfer required; the modeling in Section 3.4 uses $\lambda_2^{(K)}$ to compute the conditional bulk probability.

3.4.1 Baselines and Speedup Conditions

We compare the expected latency of three reference schemes and derive simple thresholds.

Always send full state (baseline). Sending the payload every time yields

$$L_{\text{full}} = \frac{n_{\text{data}}}{B},$$

that is, we ignore baseline compute cost when transmitting full state.

Single-object ID. As above,

$$L_{\text{ID}} = T_{\text{encode}} + T_{\text{verify}} + \frac{n_{\text{ver}}}{B} + p_{\text{desync}}(1 - p_{\text{miss}})\frac{n_{\text{data}}}{B}.$$

ID is faster than always sending when $L_{\text{ID}} < L_{\text{full}}$, i.e.,

$$T_{\text{encode}} + T_{\text{verify}} + \frac{n_{\text{ver}}}{B} \;<\; \Big[1 - p_{\text{desync}}(1 - p_{\text{miss}})\Big]\frac{n_{\text{data}}}{B}.$$

Equivalently, for a given platform and bandwidth, the maximum per-query compute budget that still yields speedup is

$$T_{\text{encode}} + T_{\text{verify}} \;<\; \Big[1 - p_{\text{desync}}(1 - p_{\text{miss}})\Big]\frac{n_{\text{data}}}{B} - \frac{n_{\text{ver}}}{B}.$$

K-ID (batch OR over K candidates). Let $T^{(K)}_{\text{encode}}$ and $T^{(K)}_{\text{verify}}$ denote the batch encode/verify costs. In practice we observe $T^{(K)}_{\text{encode}} \approx T_{\text{encode}}$ (one tag per query) and $T^{(K)}_{\text{verify}} = \alpha + \beta K$ for constants (α, β) measured on the platform (per-candidate inner-loop cost). Let q_{hit} be the probability at least one of the K candidates truly matches (a semantic acceptance event). The expected latency becomes

$$L_{\text{KID}} = T^{(K)}_{\text{encode}} + T^{(K)}_{\text{verify}} + \frac{n_{\text{ver}}}{B} + \underbrace{(1 - q_{\text{hit}})(1 - \lambda_2^{(K)})}_{\text{no hit and no false accept}} \frac{n_{\text{data}}}{B}.$$

Relative to K repeated single-object checks (serial queries), batch K-ID eliminates $K - 1$ verifier round-trips and $K - 1$ encode/verify pairs, which is beneficial when

$$T^{(K)}_{\text{verify}} + \frac{n_{\text{ver}}}{B} \;\ll\; K\Big(T_{\text{verify}} + \frac{n_{\text{ver}}}{B}\Big),$$

and the increase in $\lambda_2^{(K)}$ (cf. $1 - (1 - p)^K$) still keeps the conditional bulk probability low in the operating regime. This inequality captures the typical edge case where computation and small messages dominate per-query overheads.

3.5 Measurement Protocols

We measure compute times and false-accept probabilities with the following reproducible protocols.

3.5.1 Compute time measurements

Compute times are measured using Python's `time.perf_counter()` in a controlled experiment harness under `experiments/`. The harness supports these options that affect results and are explicitly recorded:

- **Warmup**: perform W warmup iterations before timed runs to let caches and JITs (if any) stabilize. We record runs with and without warmup to expose sensitivity.
- **Pinning**: the process can be pinned to a single CPU core using `taskset` to reduce scheduling noise.
- **Per-iteration sampling**: we record either all iteration latencies or a sampled subset; percentiles (5/50/95) are reported.

All timing scripts output CSV rows with columns: `id_system`, `n_ver`, `n_data`, `warmup`, `iteration`, `elapsed_s` (or aggregated statistics). Example scripts: `experiments/measure_encoder_time.` and `experiments/measure_encoder_time_samples.py`.

3.5.2 False-accept / Monte-Carlo estimation

Estimating very small false-accept probabilities requires Monte-Carlo sampling of desynchronization events. The procedure is:

1. Fix an ID configuration (encoder parameters, tag positions, n_{ver}).
2. Repeat N trials: generate a random desync pattern (or select adversarial messages if doing worst-case analysis), run the verifier against a non-matching object, and record whether the verifier incorrectly accepts.
3. The empirical false-accept probability is $\hat{p} = m/N$ where m is the number of observed false accepts. We compute Wilson-score 95% confidence intervals for small $\hat{p}$ when needed.

For configurations with extremely rare events (e.g., RSID with parameters that drive p below 10^{-4}), we run up to $N = 10^6$ trials as budget allows. Scripts store raw counts and CSVs (see `experiments/results/fig2_results.csv`).

3.6 Parameter Sweep and Plots

The experimental sweep varies these parameters to reproduce the baseline figures and to characterize latency:

- Field/exponent parameters (e.g., RSID `gf_exp` values mapping to `n_ver` in bits), typically values corresponding to the 12 plotted points in Fig.2.
- Verifier lengths n_{ver} (bytes) and payload sizes n_{data} (we report results for 96 bytes and 4001 bytes as representative small and large payloads).
- Bandwidths $B \in \{1, 10, 100\}$ Mbps for latency curves (converted to bytes/s in code).

Plots produced in this chapter's experiments include:

- False-accept probability vs verifier length (reproduction of Fig.2).
- Verification compute time vs verifier length.
- Modeled expected latency vs n_{ver} for multiple bandwidths and for $n_{\text{data}} = 96$ and 4001 (these appear in Chapter 5).

3.7 Reproducibility and Implementation Notes

All scripts and raw CSV outputs used to create figures are included under the `experiments/` directory. Key reproducibility notes:

- The Python environment is an editable install of the repository; dependencies are recorded in `pyproject.toml` and the `venv` used is documented in the project README.
- Native bindings (the `ecidcodes` package and `libIdcodesLibrary.so`) are required to run the ID encoders. If unavailable, the harness falls back to a pure-Python stub for functional reproduction (see `src/idsys/core/common.py`).

- For timing-sensitive microbenchmarks we recommend pinning the process and disabling frequency scaling where possible; the harness supports `taskset` and prints the CPU affinity used.

3.8 Summary

This chapter states the latency model and the measurement protocols used across experiments. The model (compute + conditional bulk transfer) is intentionally simple yet captures the key trade-off: reducing p_{miss} by increasing verifier size or multiple tags reduces conditional bulk traffic but may increase compute latency. The next chapter presents experimental results that instantiate the model and quantify these tradeoffs on our platform.

4 Implementation

hah

5 Results and Evaluation

hah

6 Conclusion and Future Work

11

Bibliography

[1] Caspar von Lengerke et al. “Stopping the Data Flood: Post-Shannon Traffic Reduction in Digital-Twins Applications”. In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. 2022, pp. 1–5. DOI: 10.1109/NOMS54207.2022.9789759.

[2] Mohammad Javad Salariseddigh et al. “Deterministic K-Identification for Future Communication Networks: The Binary Symmetric Channel Results”. In: *Future Internet* 16.3 (2024). ISSN: 1999-5903. DOI: 10.3390/fi16030078.

[3] Ievgenii A. Tsokalo et al. “Remote Robot Control with Human-in-the-Loop over Long Distances Using Digital Twins”. In: *2019 IEEE Global Communications Conference (GLOBECOM)*. 2019, pp. 1–6. DOI: 10.1109/GLOBECOM38437.2019.9013428.

A Appendix

11